

Mission 005 Backend Approach

Goal

Implement a single canonical database architecture where agents write through a backend API and the application reads from the same database through read APIs.

Approach

Replace the snapshot-first pattern with a direct write-to-DB pattern. The backend should expose a write endpoint for agent events, persist those events in the canonical database, and continue to serve the dashboard through read-only APIs that query the same database. The frontend remains consumer-only and should never become the source of truth.

Backend responsibilities

- Design and own the canonical database schema
- Expose a write API for agent events and mission state updates
- Validate write requests
- Persist canonical mission state in the database
- Expose read APIs for dashboard and latest mission data
- Preserve stable ordering and freshness fields
- Keep read responses predictable for the UI
- Add backend tests for write validation, persistence, and read contract stability

Data model responsibilities

- Represent missions, actors, events, timestamps, sequence ordering, and freshness metadata
- Preserve enough structure for chronological rendering and parallel activity grouping
- Avoid duplicating the same source-of-truth data in multiple stores
- Keep any JSON export strictly secondary

Migration / bootstrap approach

- Use Neon as the canonical production and preview database through the user's Vercel setup
- If the current live source contains useful state, backfill it into the canonical database once
- After backfill, writes should go only to the backend API and DB
- If there is a temporary transition period, make it explicit and time-boxed
- Keep the dashboard read path stable during migration

Read path

- ``GET /api/dashboard`` should query the canonical database directly or through a thin service layer

- `GET /api/missions/latest` should continue returning the latest mission view
- Freshness / lag metadata should be derived from the DB state and timestamps

Write path

- Agent clients submit events to a backend endpoint
- The backend validates payload shape, schema, and ordering assumptions
- The backend stores the event and any derived mission state in the DB
- The backend returns a clear success response so the agent knows the event is canonical

Verification targets

- A write request from an agent persists in the canonical DB
- The dashboard API reflects the new state without JSON replication as the main path
- The UI updates from the read API path
- Response shape remains stable
- Backend tests cover write, read, and migration behavior